

Company Data Enrichment - Code Explanation

1. Module Goal

company_data_enrichment is the preprocessing pipeline for Entity Resolution. It reads the company CSV, collects official and social URLs, crawls the web with Crawl4AI, extracts evidence, validates country/city fields, merges enriched values, and exports a compact data_er_ready.csv file for the next ER stage.

The current codebase does not use Spark. Local processing is implemented with pandas, Crawl4AI, BeautifulSoup, and project-specific Python rules.

Main design choices

- Keep target_fields as the canonical schema, then map many web formats into that schema.
- Crawl4AI remains the primary crawler. BeautifulSoup is used only to parse crawled HTML, JSON-LD, and meta tags.
- Country code is normalized as ISO-2, for example main_country_code = US and main_country = United States.
- Strong TLDs such as .dk, .se, and .de can lock country/country_code early, while crawling still continues for all other fields.
- Official websites are deep crawled. Social URLs stay in the manifest but are crawled only as single seed pages.
- The crawler now runs in batches so one Playwright/Chromium context does not need to survive thousands of URLs.

2. File Structure

File	Responsibility
config/company_data_enrichment.yaml	Input/output paths, URL columns, crawl settings, validation settings, ER-ready thresholds, and target_fields.
src/company_data_enrichment/pipeline.py	Main orchestrator: build-manifest, crawl, extract, merge, export-er-ready, run-all.
src/company_data_enrichment/crawler.py	Crawl4AI wrapper: browser config, deep crawl, URL filtering, batch restart, result flattening.
src/company_data_enrichment/extractors.py	HTML/markdown extraction via BeautifulSoup, JSON-LD, meta tags, regex email/phone/year/employee.
src/company_data_enrichment/geo_signals.py	Country/city inference, ISO normalization, TLD/language/text signals, voting verdicts.
src/company_data_enrichment/url_manifest.py	URL normalization and manifest/queue generation.
src/company_data_enrichment/rules.py	Rule engine for KEEP, ADD, REPLACE, and CONFLICT_REVIEW.
src/company_data_enrichment/pandas_io.py	CSV and Parquet read/write helpers.
src/company_data_enrichment/quality_report.py	Quality report generation after merge.
test/	Unit tests for crawler, extractor, geo signals, pipeline, rules, and I/O.

3. Input and Configuration

The primary input is dataset/data_sample_AofAI.csv. It contains the original company fields, main_* fields, website_url, website_tld, website_language_code, and social URLs such as linkedin_url, facebook_url, instagram_url, twitter_url, and youtube_url.

```

input_csv: dataset/data_sample_AofAI.csv
output_dir: dataset/silver

url_columns:
  website_url: official
  linkedin_url: social
  facebook_url: social
  instagram_url: social
  twitter_url: social
  youtube_url: social

crawl:
  max_concurrency: 3
  crawl_batch_size: 100
  timeout_ms: 60000
  retry_count: 2
  deep_crawl_enabled: true
  deep_crawl_max_depth: 1
  deep_crawl_max_pages: 5
  deep_crawl_include_external: false

```

Crawl config	Meaning
max_concurrency: 3	Number of URLs/pages crawled in parallel within one batch. A value of 3 improves speed while avoiding too many Chromium tabs at once.
crawl_batch_size: 100	Number of seed URLs processed before restarting AsyncWebCrawler/Chromium context. A value of 100 reduces long-lived context errors such as pipe closed by peer or Browser.close failure.
timeout_ms: 60000	Maximum wait time for one page crawl, in milliseconds. 60000 means 60 seconds before treating the page as timed out.
retry_count: 2	Number of retries Crawl4AI may attempt after a URL fails. Useful for temporary failures, but slower when a website is blocked or offline.
deep_crawl_enabled: true	Enables deep crawling. In this codebase only official websites are deep crawled; social URLs are forced into single-page mode to reduce blocking.
deep_crawl_max_depth: 1	Maximum internal-link depth from the seed URL. Depth 1 means the seed page plus direct internal links such as about/contact/product.
deep_crawl_max_pages: 5	Maximum pages per official seed URL. A value of 5 usually means the seed page plus up to 4 related internal pages.
deep_crawl_include_external: false	Do not follow links to external domains. This avoids crawling social networks, CDNs, ads, tracking pages, or unrelated websites.

target_fields is the canonical enrichment schema. Extractors may read values from JSON-LD, meta tags, body text, markdown, or CSV signals, but all evidence is mapped back into these field names.

```

target_fields:
  - company_name
  - company_legal_names
  - company_commercial_names
  - main_country_code
  - main_country
  - main_region
  - main_city
  - main_postcode
  - main_street
  - primary_email
  - primary_phone
  - short_description
  - year_founded
  - employee_count

```

4. How to Run

```

PYTHONPATH=src python -m company_data_enrichment.pipeline build-manifest
PYTHONPATH=src python -m company_data_enrichment.pipeline crawl
PYTHONPATH=src python -m company_data_enrichment.pipeline extract
PYTHONPATH=src python -m company_data_enrichment.pipeline merge
PYTHONPATH=src python -m company_data_enrichment.pipeline export-er-ready

# Full pipeline:
PYTHONPATH=src python -m company_data_enrichment.pipeline run-all

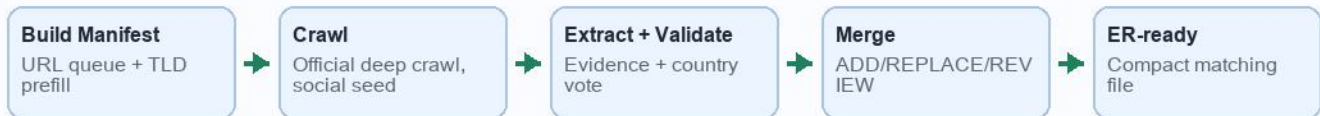
# Small smoke run:
PYTHONPATH=src python -m company_data_enrichment.pipeline run-all --limit 10

# Tests:
PYTHONPATH=src pytest -q

```

5. Detailed Flow

Overall Processing Flow



Outputs:

url_manifest, crawl_results_raw, crawl_extracted_fields, company_validation_results, data_enriched, data_er_ready.

Step 1: build-manifest

run_build_manifest reads the source CSV and writes url_manifest.parquet, crawl_queue.parquet, and geo_prefill.parquet. The manifest preserves the relationship between company ID, URL, source type, source column, and URL priority. The queue is the deduplicated list of seed URLs to crawl.

- official: website_url, eligible for deep crawling.
- social: LinkedIn/Facebook/Instagram/Twitter/YouTube, crawled as seed pages only.
- geo_prefill: if website_tld or website_url has a strong country-specific TLD, the pipeline creates locked evidence for main_country_code and main_country.

Example: website_url = https://example.dk or website_tld = dk creates main_country_code = DK and main_country = Denmark. This evidence is locked because .dk is country-specific. Crawling still continues to enrich city, email, phone, description, year_founded, and employee_count.

Step 2: crawl

run_crawl reads crawl_queue.parquet and calls Crawl4AI. Official URLs use BFSDeepCrawlStrategy with depth 1 and a maximum of 5 pages per seed by default. Social URLs are forced into single-page mode, so they usually have depth = 0. The crawler now has crawl_batch_size = 100. This restarts AsyncWebCrawler per batch so the Playwright/Chromium context does not need to stay alive for 5000+ URLs. If a social/YouTube page destabilizes the context, other batches can still keep their results.

```

CrawlerRunConfig:
    wait_until = domcontentloaded
    delay_before_return_html = 2.0
    page_timeout = 60000
    remove_overlay_elements = True
    remove_consent_popups = True
    simulate_user = True
    override_navigator = True
    magic = True
    scan_full_page = True
    max_scroll_steps = 3
    max_retries = 2

```

crawl_results_raw.parquet contains: seed_url, canonical_url, domain, depth, source_type, source_col, priority, success, fetched_at, markdown, html, metadata_json, and error_message.

DNS failures, anti-bot pages, CAPTCHA blocks, LinkedIn/YouTube failures, and BrowserContext errors are stored as `success=False` rows. The pipeline does not crash because one URL fails.

Step 3: extract

`run_extract` reads `crawl_results_raw.parquet`. `extractors.py` parses HTML with BeautifulSoup, removes `script/style/nav/footer/header/aside` from clean text, and extracts field-level evidence.

- JSON-LD Organization/Corporation/LocalBusiness: name, legalName, email, telephone, address, foundingDate/foundingYear, numberOfEmployees.
- Meta tags: title, description, geo.region, geo.placename, og:locale, language/html lang.
- Plain text/markdown: regex extraction for email, phone, founded year, employee count, country/city text signals.
- A failed crawl row can still be extracted if HTML or markdown is present.

`run_extract` also creates CSV-based evidence from `website_tld` and `website_language_code`. TLD evidence is stronger than language evidence. Then `build_company_validation_results` creates `company_validation_results.csv/parquet` from the current input CSV plus crawl evidence. It does not read `candidate_validation_v2.csv`.

Step 4: country validation

`geo_signals.py` combines country signals from TLD, language, JSON-LD, meta/text evidence, and social evidence. Signals are voted with weights. Strong TLD and JSON-LD are usually stronger than language/html locale.

Verdict	Meaning
CONFIRMED	The original country matches the voted country.
MISSING_FILLED	The original country is missing and validation is confident enough to fill it.
CONFLICT_TLD	A country-specific TLD conflicts with the original country and suggests correction.
CONFLICT	Web/JSON-LD/text evidence conflicts with the original country with enough confidence.
CONFLICT_UNCERTAIN	Signals conflict but confidence is low, so manual review is preferred.
insufficient_data	There is not enough validation evidence.

The weak locale policy is configured in YAML. For example, `en`, `en-US`, and `english` can be treated as weak signals unless supported by stronger evidence, preventing most English websites from being inferred as United States.

Step 5: merge

`run_merge` selects the best evidence for each `Applications_of_AI_id` and field. Evidence is sorted by locked country rank, `final_confidence`, `source_rank`, `depth`, and URL priority.

```
final_confidence = min(field_confidence + source_bonus, 1.0)
source_bonus: official/validation +0.20, csv/social +0.00
source_rank: validation -> official -> csv -> social -> other
Tie-breakers: locked country first, higher confidence, lower depth, lower URL priority
```

`rules.py` assigns actions per field: KEEP when no evidence exists or the value matches the original; ADD when the original is missing and confidence passes `add_threshold`; REPLACE when the value differs and confidence passes `replace_threshold`; CONFLICT_REVIEW when the value differs but confidence is not strong enough.

Step 6: export-er-ready

`data_enriched.csv` is the full audit dataset. `data_er_ready.csv` is the compact file for Databricks/ER. ER-ready keeps `input_company_name` and `input_main_*` fields for audit while producing normalized fields for matching.

- `company_name`: keep the original if present; use `final_company_name` only when the original is missing.
- `main_country_code/main_country`: prefer accepted validation results with enough confidence.
- `main_city`, `primary_email`, `short_description`: use field-specific thresholds from `er_ready` config.
- `year_founded` and `employee_count`: add-only. Keep original if present; fill only if original is missing and confidence is sufficient.

6. Output Files

Output	Content
dataset/silver/url_manifest.parquet	All normalized URL seeds with source_type, source_col, priority, and company ID.
dataset/silver/crawl_queue.parquet	Deduplicated queue for the crawler.
dataset/silver/geo_prefill.parquet	Locked country evidence from strong TLDs.
dataset/silver/crawl_results_raw.parquet	Raw HTML/markdown/metadata/error rows per crawled page.
dataset/silver/crawl_extracted_fields.parquet	Long-form evidence: each row is one extracted field.
dataset/silver/company_validation_results.csv	Country/city validation report with signals, vote, verdict, and recommended_action.
dataset/silver/data_enriched.csv	Full enriched dataset with final_* columns, action columns, evidence_json, and actions_json.
dataset/silver/quality_report.parquet	Action counts by field for quality audit.
dataset/silver/data_er_ready.csv	Compact clean file for Databricks/ER matching.

7. Example Flow

Input row: company_name = ABC A/S, main_country_code missing, website_url = https://abc.dk, and linkedin_url available.

- build-manifest creates an official seed from website_url and a social seed from linkedin_url.
- geo_prefill sees .dk and creates locked evidence: main_country_code = DK, main_country = Denmark.
- crawl still crawls the official website for about/contact pages and tries the LinkedIn seed page.
- extract finds info@abc.dk and founded year 2005 from JSON-LD/body text.
- merge adds country, email, and year if missing. If crawled company_name differs from the original, the full enriched file can mark it as REPLACE or CONFLICT_REVIEW, but ER-ready keeps the original company_name when present.
- export-er-ready writes one compact row for matching, including country_validation_verdict and source columns.

8. Common Crawl Issues

- Execution context was destroyed: usually a page redirected/reloaded while Crawl4AI was trying to remove a consent popup. If the page still completes successfully, it can be ignored.
- BrowserContext.new_page / pipe closed by peer: the Playwright/Chromium context was closed. The current code reduces this risk with crawl_batch_size and batch-level context restart.
- LinkedIn/Facebook/YouTube failures are common because social sites have strong anti-bot protections. The pipeline keeps social URLs but does not deep crawl them.
- Cloudflare JS challenge: the website is protected by Cloudflare anti-bot checks. Crawl4AI may retry but still be blocked. This is a website protection mechanism, not a pipeline logic bug.
- net::ERR_NAME_NOT_RESOLVED: DNS cannot resolve the domain, for example because the domain expired, the URL is wrong, the website is offline, or the network cannot find the host. The URL is written as a failure row and the pipeline continues with other evidence sources.

Example DNS failure:

```
[ANTIBOT]. http://egeneration.com.bd | Error: Proxy direct failed:
Page.goto: net::ERR_NAME_NOT_RESOLVED at http://egeneration.com.bd/
```